MUSLIMOS BRAND GUIDE ALIGNMENT

MuslimOS

Android Play Store Launch Guide

A practical, brand-aligned roadmap for turning MuslimOS into a proper Android app on the Play Store without wrecking the current product quality.

RECOMMENDED PATH

Keep the current app, package it with Capacitor, ship Android first, and only add SQLite when offline content scale or search genuinely demands it.

Prayer times · Quran · Hadith · Dhikr · Qibla

March 2026

بِسْمِ اللَّهِ

EXECUTIVE SUMMARY

Ship MuslimOS to Android without losing the product's elegance.

The right move here is disciplined, not dramatic: protect the current app, harden it for mobile, and only introduce heavier architecture when the product earns it.

BEST FIT

Capacitor + current app

Fastest route to release, lowest rewrite risk, and the cleanest way to preserve the UI quality already built into MuslimOS.

SQLITE

Optional, not mandatory

Useful later for heavy offline content and structured storage. Not a reason to block a first Android release.

MUSLIMOS RECOMMENDATION

Treat Android launch as a product packaging project first, not a rewrite project. Ship the current experience in a native shell, then deepen native infrastructure later.

ARCHITECTURE

Choose the least destructive path.

The elegant solution is the one that protects visual quality and shipping velocity at the same time.

PATH	WHY IT FITS	USE WHEN
Capacitor + current app	Keeps the existing Vite + React codebase alive and avoids rebuilding the UI from scratch.	You want a serious Android release soon with minimal technical waste.
Flutter rewrite	Strong native control, but it resets product velocity and forces a full UI rebuild.	You are planning a larger platform rewrite anyway.
React Native rewrite	Shared JavaScript skills, but still a major migration with a lot of integration churn.	You need deeper native UI later and accept a rewrite cost now.

STORAGE STRATEGY

Only add a database when the product truly needs one.

Don’t burden v1 with infrastructure that sounds mature but slows delivery without improving the user experience.

KEEP SIMPLE STORAGE FOR NOW

Good for v1

- settings and language
- streaks and bookmarks
- last-read position
- checklists and tasbeeh state
- light journal entries

ADD SQLITE LATER

Use when scale becomes real

- offline hadith bundles
- downloaded tafseer packs
- large search indexes
- sync-ready local data models
- heavier long-term journaling data

Phase 1: no SQLite block **Phase 2: migrate if needed** **Phase 3: optimize for scale**

IMPLEMENTATION

Move from web app to Android product in practical phases.

The sequence below is built for MuslimOS specifically: protect what already works, then add native polish deliberately.

PHASE 1

Stabilize the existing app for mobile shell use

Audit mobile viewport quality, audio behavior, Android back handling, safe areas, and persistence across all major flows.

PHASE 2

Add Capacitor and Android Studio release flow

Package the built app, configure Android, and establish a proper release pipeline around signed bundles.

PHASE 3

Add native polish where users feel it

Splash, icons, status bar, notifications, and permission flows should feel deliberate and trustworthy.

PHASE 4

Ship internal, then public

Use internal and closed testing first, then move to production once crash-free behavior and QA confidence are strong enough.

```
npm install @capacitor/core @capacitor/cli
npx cap init
npx cap add android
npm run build
npx cap sync android
npx cap open android
```

NATIVE SCOPE

Add native polish where Android users will actually feel it.

This is the layer that makes MuslimOS feel like an app, not a website inside a frame.

HIGH PRIORITY

What should be native early

- splash screen
- icons and launch branding
- status bar styling
- Android back behavior
- local notifications

OPTIONAL LATER

Only when product scope grows

- SQLite-backed offline content
- background jobs
- sync/account architecture
- biometric privacy for journal

QA

Do not submit a worship app with unstable basics.

MuslimOS should feel calm, clear, and dependable on real Android devices.

□	Home, Quran, Worship, Hadith, and drawer navigation all behave correctly on device
□	Surah and ayah audio remain stable across background, resume, pause, and seek
□	Tafseer language follows the global app language every time
□	Islamic Calendar and Hijri date resolve correctly for the user's local region
□	Journal and Worship tracker persist local state without loss
□	Qibla permission failures show a graceful and understandable fallback
□	No horizontal scroll, broken safe areas, or awkward bottom-nav collisions on Android

PLAY STORE

Prepare the release like a product, not just a build artifact.

The listing itself is part of the MuslimOS brand experience.

REQUIREMENT	WHAT YOU NEED	WHY IT MATTERS
Build	Signed Android App Bundle (.aab)	Required for Play Store release
Assets	Icon, screenshots, feature graphic, store copy	Defines first impression and trust
Compliance	Privacy policy, Data Safety, content rating	Required because the app uses location, notifications, and personal content
Rollout	Internal test → closed test → controlled production release	Reduces risk before public scale

The most elegant move

Keep the current MuslimOS app. Package it with Capacitor. Ship Android first. Add SQLite only when offline structured data genuinely becomes a product requirement. That path is technically sound, fast to market, and aligned with the quality already present in the product.